

in principle however other distance metrics can also be used. In the final step, only those neurons which lie in the neighbourhood of the winning neuron, defined by $h(l_i, l^*)$ are activated and the weights w_i corresponding to those neurons are updated as follows:

$$w_i^{t+1} = w_i^t + \alpha^t h^t(x - w_i^t), \quad (3)$$

where, $h^t(l^*, l_i)$ is the neighbourhood function and α^t is the learning rate. The explicit dependance of t in both the learning rate α and neighborhood function $h(l^*, l_i)$ reflects the fact that these are iteration dependent quantities and are usually artificially reduced in magnitude at each iteration. Intuitively, the neurons that are closest to the BMU are updated in the direction of the sample x , whereas the neurons which are far away from BMU are updated the least. Along with the learning rate α , the form of the neighborhood function is a hyperparameter of the algorithm, but usually the Gaussian functional form

$$h(d_{ij}) = \exp\left(-\frac{d_{ij}^2}{2\sigma^2}\right), \quad (4)$$

is preferred. Here, d_{ij} refers to the distance between the nodes l_i and l_j . The completion of the training process results in a map $\kappa(x)$ from the high-dimensional space Ω to the lattice space of neurons L , which preserves the topological structure present in Ω . During the inference phase, the weights of the neural network are not updated. Instead, the appropriate BMU l^* is picked for a given data sample x .

In contrast to the alternative methods of dimensionality reduction, SOM exhibits non-parametric and non-linear mapping characteristics. The algorithm is robust, the learning is time-efficient, and the outlines of the map are developed quickly [50].

B. Kernelized self-organizing map

Kohonen's original method was refined by Andras by integrating scenarios in which the original data vector must be projected into a higher-dimensional feature space in order to accurately identify the best matching unit [51]. Kernel methods, a well-known concept in the field of machine learning, are used in this improved algorithm.

Kernel approaches are prevalent in pattern recognition, machine learning, and artificial intelligence [52]. They are very useful for distinguishing between non-linearly separable classes of data points. The fundamental principle behind kernel methods is that the Euclidean inner product between two data vectors (as seen in the preceding algorithm) can be replaced with an equivalent inner product in a higher-dimensional feature space. The simple substitution

$$x_i \cdot x_j \rightarrow k(x_i, x_j) = \phi(x_i) \cdot \phi(x_j), \quad (5)$$

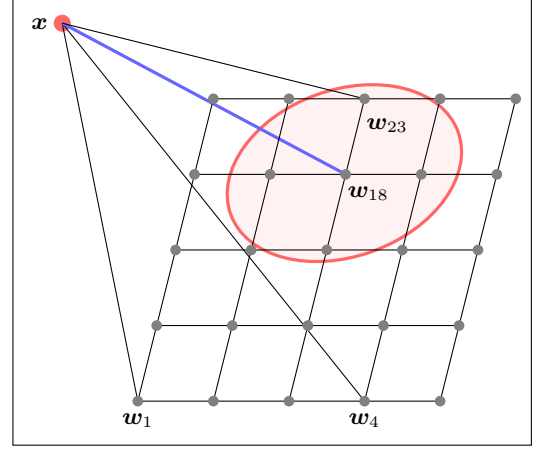


FIG. 1. A snapshot of the (classical) self-organizing map during the training phase. First, the weights are randomly initialized and then the best matching unit (BMU) is found (here, l_{18}) by calculating the Euclidean distance between x and all the weights $\{w_1, \dots, w_k\}$, see Eq. (2). The weights in the neighbourhood (here represented by a red circle) of the winning neuron are updated using the update rule specified in Eq. (3).

essentially allows one to explore the higher dimensional spaces $\phi(x)$ without explicitly constructing the mapping between the original Euclidean space to the higher dimensional space. Kernel approaches excel when the inner product of two data vectors can be calculated more efficiently than the calculation of the explicit higher dimensional map.

Andras observed that the updates of the weight vectors do not need to take place in higher dimensional spaces. The observations made were as follows: In Kohonen's original implementation, the update rule for the weights in the self-organizing map has an alternative interpretation of the minimization of the $\|x - w_i\|^2$, which can be seen as:

$$w_i^{t+1} = w_i^t - \bar{\alpha}^t h^t \frac{\partial}{\partial w_i} [\|x - w_i^t\|^2]. \quad (6)$$

Now, consider the mapping of a data vector x to a higher dimensional space $\Psi(x) \in \Omega$. All the weight vectors w_i 's will also be mapped to the same higher dimensional space $\Psi(w_i) \in \Omega$. The next step consists of minimizing the distance between the data vector and the weight vectors mapped to the higher dimensional spaces, i.e.

$$\|\Psi(x) - \Psi(w_{i^*}^t)\| = \min_i \{\|\Psi(x) - \Psi(w_i^t)\|\}. \quad (7)$$

The distance between the two higher dimensional vectors can be expressed in terms of the kernel function as

$$\begin{aligned} \|\Psi(x) - \Psi(w)\|^2 &= \langle \Psi(x) - \Psi(w), \Psi(x) - \Psi(w) \rangle \\ &= K(x, x) + K(w, w) - 2K(x, w), \end{aligned} \quad (8)$$